

Image Classification

Image Classification



This image by Nikita is
licensed under [CC-BY 2.0](#)

(assume given a set of labels)
{dog, cat, truck, plane, ...}



cat

The Problem: Semantic Gap



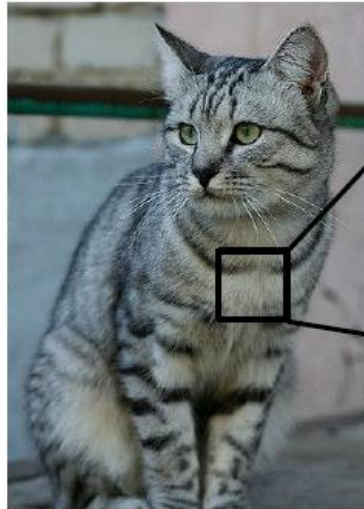
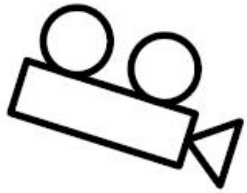
This image by Nikita is
licensed under [CC-BY 2.0](#)

[	105	112	108	111	104	99	106	99	96	103	112	119	104	97	93	87]
[	91	98	102	106	104	79	98	103	99	105	123	136	110	105	94	85]
[	76	85	90	105	128	105	87	96	95	99	115	112	106	103	99	85]
[	99	81	81	93	120	131	127	100	95	98	102	99	96	93	101	94]
[	106	91	61	64	69	91	88	85	101	107	109	98	75	84	96	95]
[	114	108	85	55	55	69	64	54	64	87	112	129	98	74	84	91]
[	133	137	147	103	65	81	80	65	52	54	74	84	102	93	85	82]
[	128	137	144	140	109	95	86	70	62	65	63	63	60	73	86	101]
[	125	133	148	137	119	121	117	94	65	79	80	65	54	64	72	98]
[	127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	84]
[	115	114	109	123	150	148	131	118	113	109	100	92	74	65	72	78]
[	89	93	90	97	100	147	131	118	113	114	113	109	106	95	77	80]
[	63	77	86	81	77	79	102	123	117	115	117	125	125	130	115	87]
[	62	65	82	80	78	71	80	101	124	126	119	101	107	114	131	119]
[	63	65	75	88	89	71	62	81	120	138	135	105	81	98	110	118]
[	87	65	71	87	106	95	69	45	76	130	126	107	92	94	105	112]
[	118	97	82	86	117	123	116	66	41	51	95	93	89	95	102	107]
[	164	146	112	80	82	120	124	104	76	48	45	66	88	101	102	109]
[	157	170	157	120	93	86	114	132	112	97	69	55	70	82	99	94]
[	130	128	134	161	139	100	109	118	121	134	114	87	65	53	69	86]
[	128	112	96	117	150	144	120	115	104	107	102	93	87	81	72	79]
[	123	107	96	86	83	112	153	149	122	109	104	75	80	107	112	99]
[	122	121	102	80	82	86	94	117	145	148	153	102	58	78	92	107]
[	122	164	148	103	71	56	78	83	93	103	119	139	102	61	69	84]

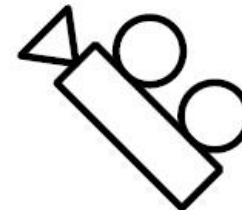
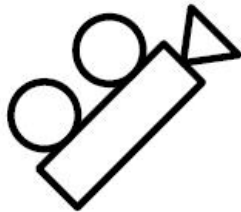
What the computer sees

e.g. 800 x 600 x 3
(3 channels RGB)

Challenges: Viewpoint variation



[	105	112	108	111	104	99	106	99	96	103	112	110	104	97	93	87]
[	91	98	102	106	104	79	98	103	99	105	123	136	110	105	94	85]
[	76	85	00	105	128	185	87	06	05	00	115	112	106	103	90	85]
[	99	81	81	93	120	131	127	100	95	98	102	99	96	93	101	94]
[	106	91	61	64	69	91	88	85	101	107	109	90	75	84	96	95]
[	114	180	85	55	55	69	64	54	64	87	112	123	98	74	84	91]
[	133	137	147	103	65	81	88	65	52	54	74	84	102	93	85	82]
[	128	137	144	140	180	05	88	78	62	65	63	63	68	73	86	101]
[	125	133	148	137	119	121	117	94	65	79	80	65	54	64	72	98]
[	127	125	131	147	133	127	126	131	111	96	80	75	61	84	72	84]
[	115	114	109	123	158	148	131	118	113	109	188	92	74	65	72	78]
[	86	93	00	97	188	147	131	118	113	114	115	109	106	95	77	80]
[	63	72	86	81	77	79	102	123	117	115	117	125	125	130	115	87]
[	62	65	82	89	78	71	88	101	124	126	119	101	107	114	131	119]
[	83	65	75	88	89	71	62	81	120	138	135	105	81	98	110	118]
[	87	85	71	87	186	95	69	45	76	138	126	187	92	84	185	112]
[	118	97	82	86	117	123	116	66	41	51	95	93	89	95	182	187]
[	104	146	112	80	82	128	124	104	76	48	45	60	88	101	182	189]
[	157	178	157	120	93	86	114	132	112	97	69	55	78	82	99	84]
[	138	120	134	161	139	188	189	118	121	134	114	87	65	53	69	86]
[	128	112	96	117	158	144	128	115	104	187	182	93	87	81	72	79]
[	123	187	06	86	83	112	153	149	122	180	184	75	88	187	112	90]
[	122	121	182	80	82	86	94	117	145	148	153	182	58	78	92	187]
[	122	164	148	183	71	56	78	83	93	183	119	138	182	61	69	84]



All pixels change when the camera moves!

[This image](#) by [Nikita](#) is licensed under [CC-BY 2.0](#)

Challenges: Background Clutter



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain

Challenges: Illumination



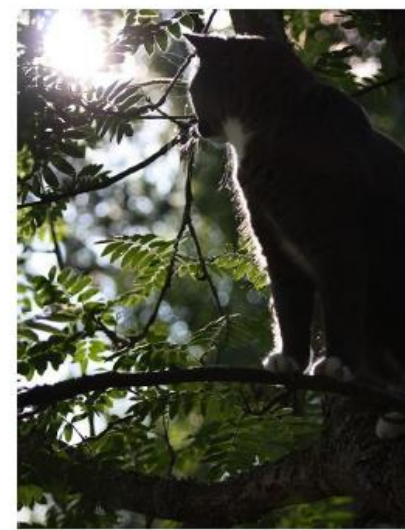
This image is CC0 1.0 public domain



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain



This image is CC0 1.0 public domain

Challenges: Occlusion



This image is CC0 1.0 public domain

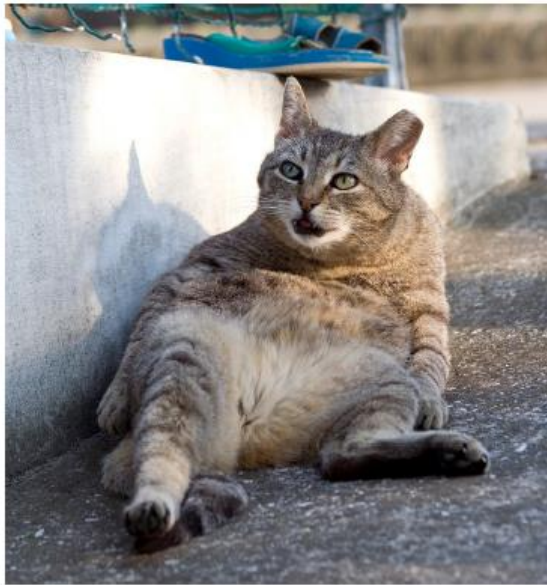


This image is CC0 1.0 public domain

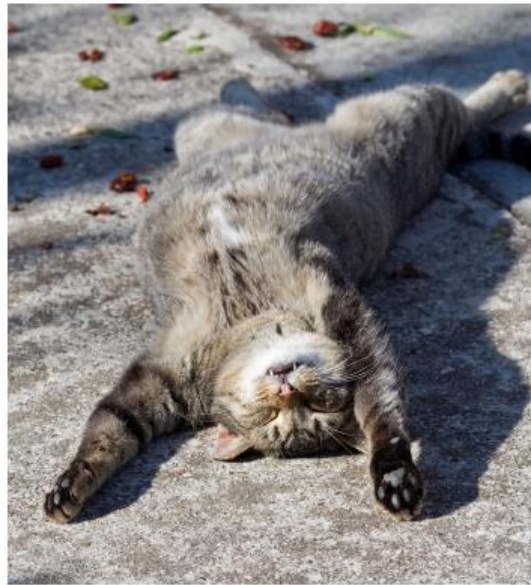


This image by ionsson is licensed under CC-BY 2.0

Challenges: Deformation



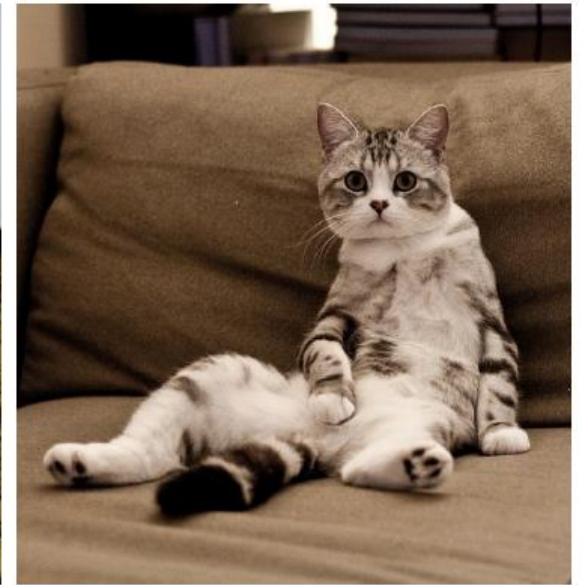
This image by Umberto Salvagnin
is licensed under CC-BY 2.0



This image by Umberto Salvagnin
is licensed under CC-BY 2.0



This image by sare hear is
licensed under CC-BY 2.0



This image by Tom Thai is
licensed under CC-BY 2.0

Challenges: Intraclass variation



This image is CC0 1.0 public domain

An image classifier

```
function [class_label] = classify_image(image)
%some code here
end
```

An image classifier

```
function [class_label] = classify_image(image)
%some code here
end
```

Unlike e.g. sorting a list of numbers, **no obvious way to hard-code** the algorithm for recognizing a cat, or other classes

Machine Learning

Unsupervised Learning

- No label or target class
- Find out properties of the structure of the data
- Clustering

Supervised Learning

Machine Learning

Unsupervised Learning

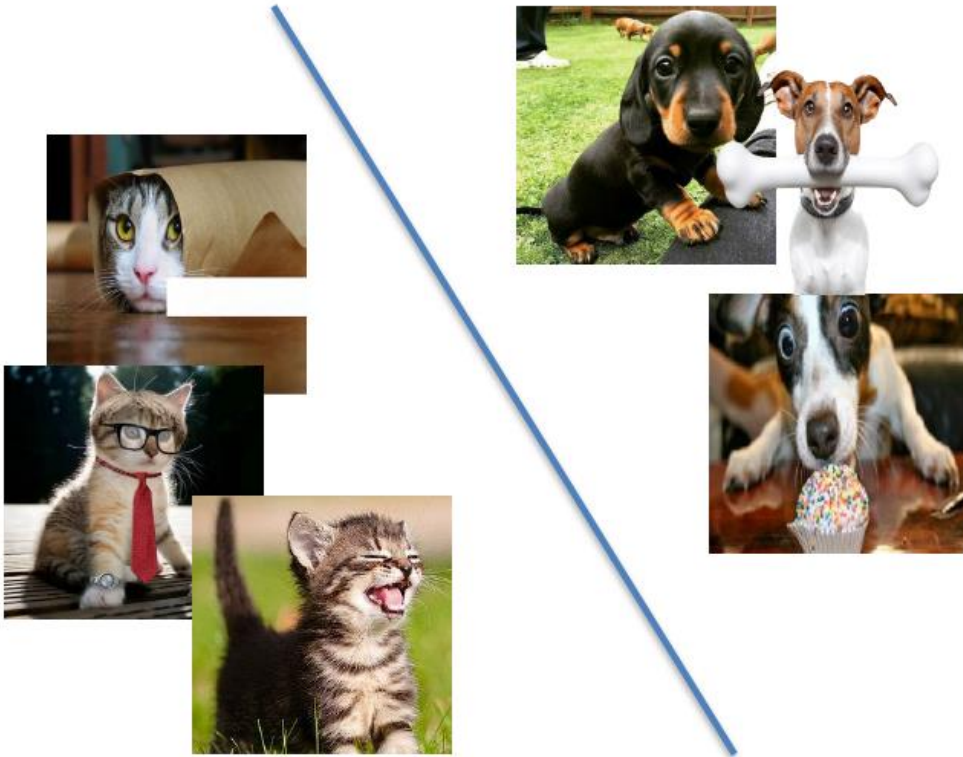


Supervised Learning

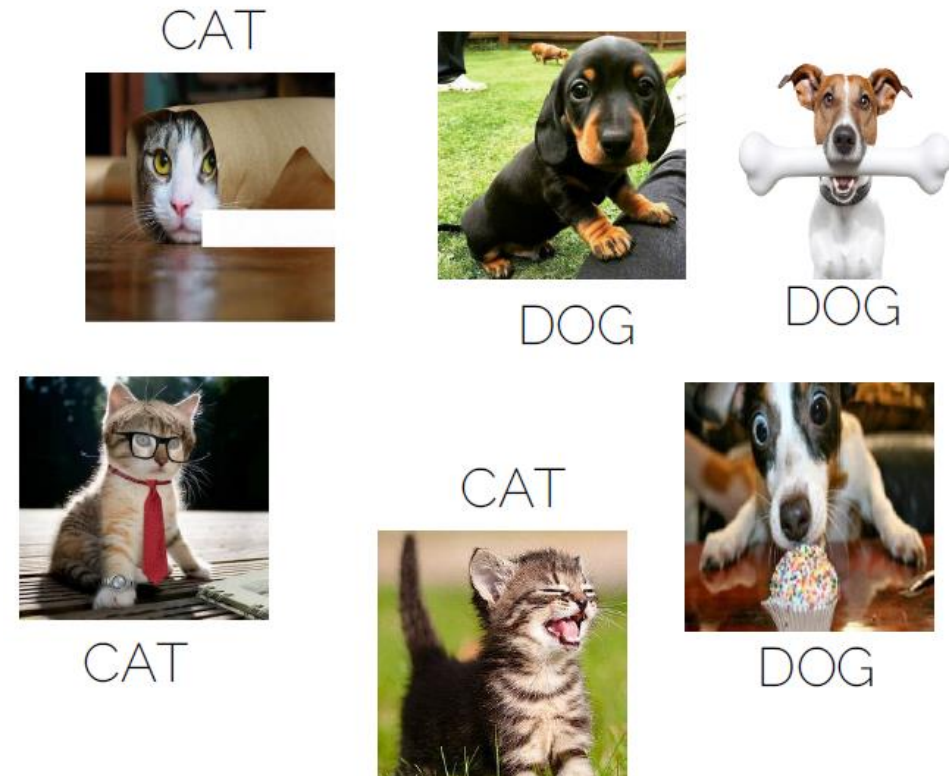


Machine Learning

Unsupervised Learning



Supervised Learning



A Simple Classifier: Nearest Neighbor

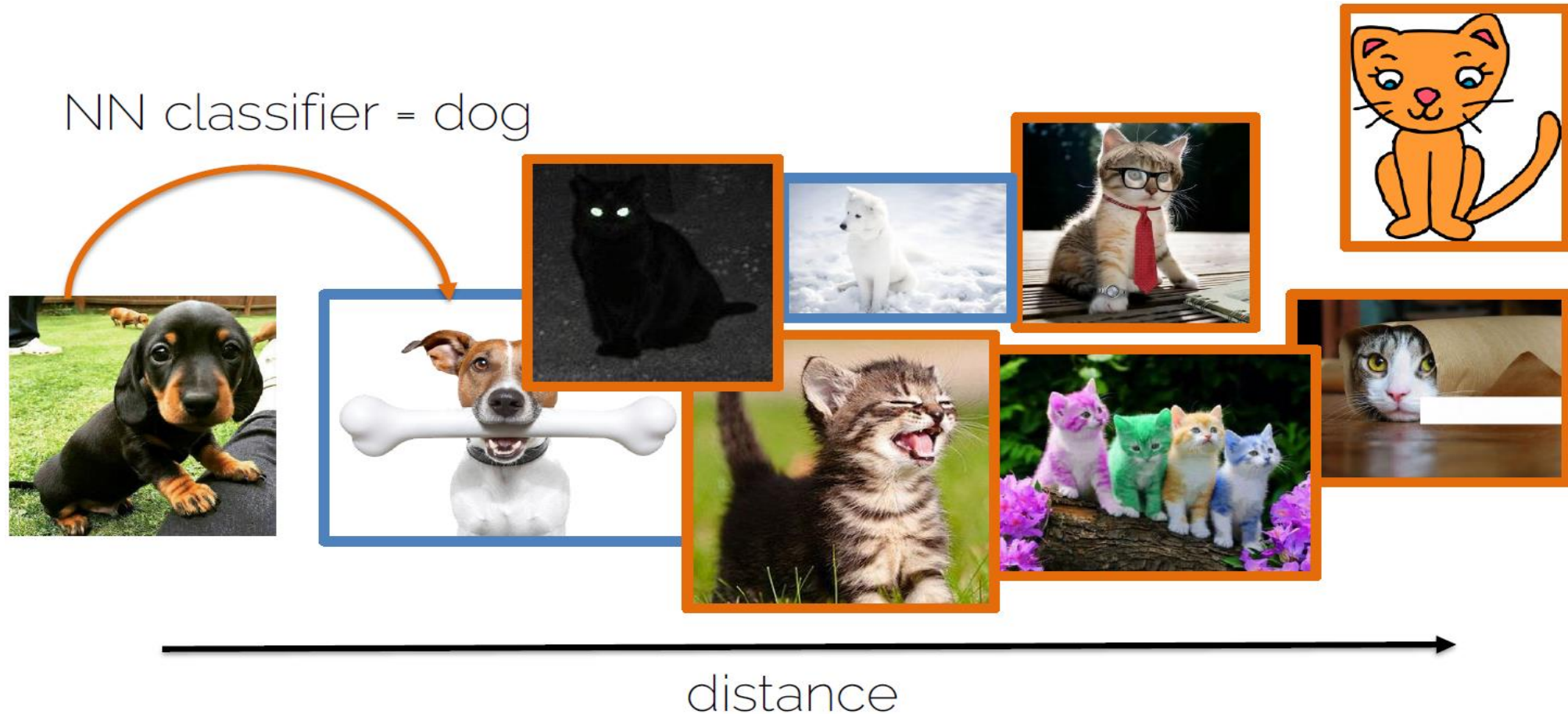
```
function [model] = train(images, labels)
%Memorize all data and labels
End
```

```
function [test_label] = predict(model, test_images)
%Predict the label of the most similar training
image
end
```

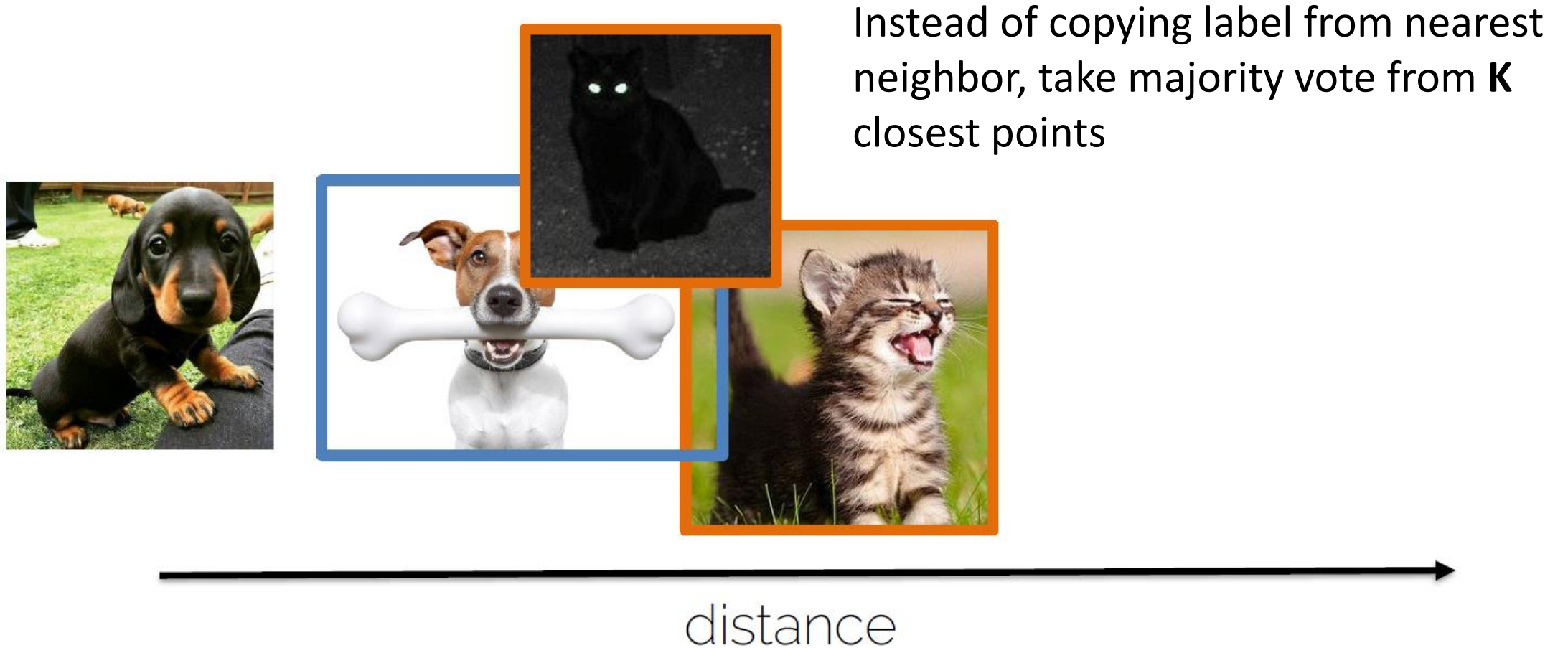
A Simple Classifier: Nearest Neighbor



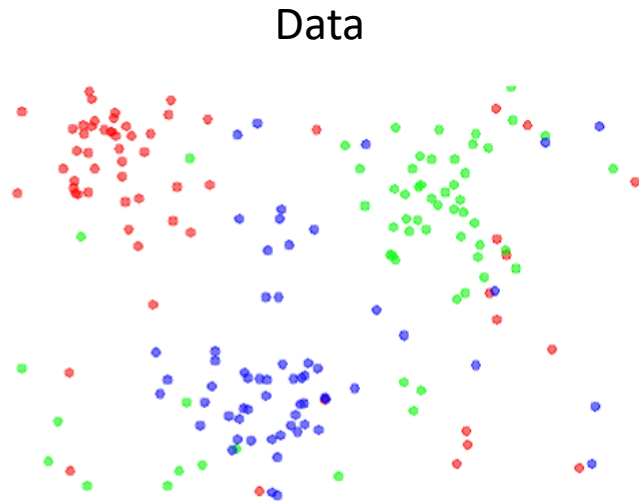
A Simple Classifier: Nearest Neighbor



A Simple Classifier: Nearest Neighbor



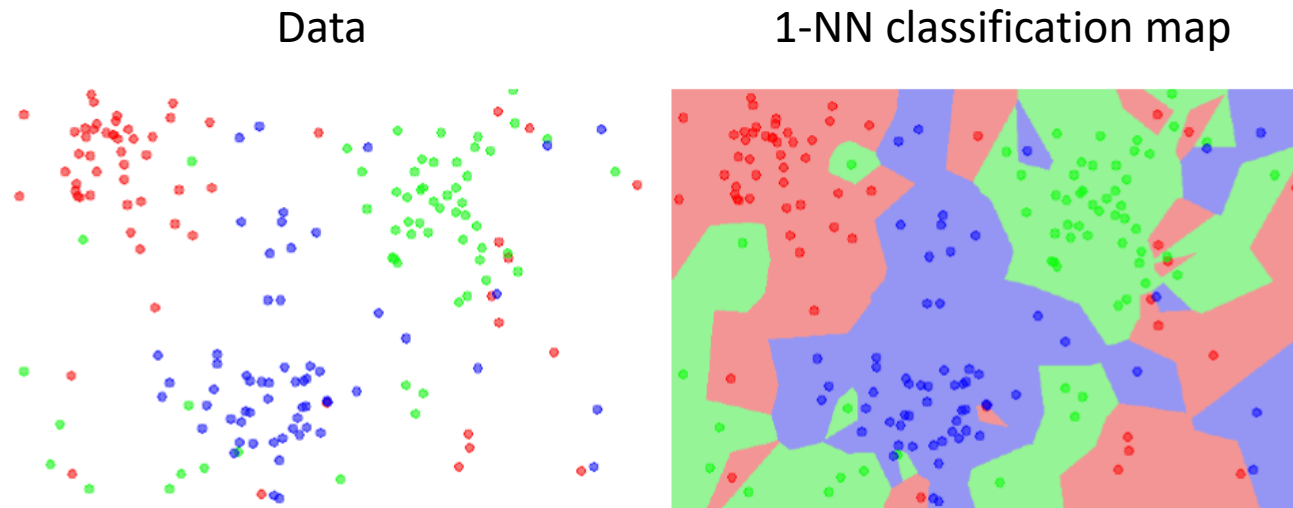
A Simple Classifier: Nearest Neighbor



Source: <https://commons.wikimedia.org/wiki/File:Data3classes.png>

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

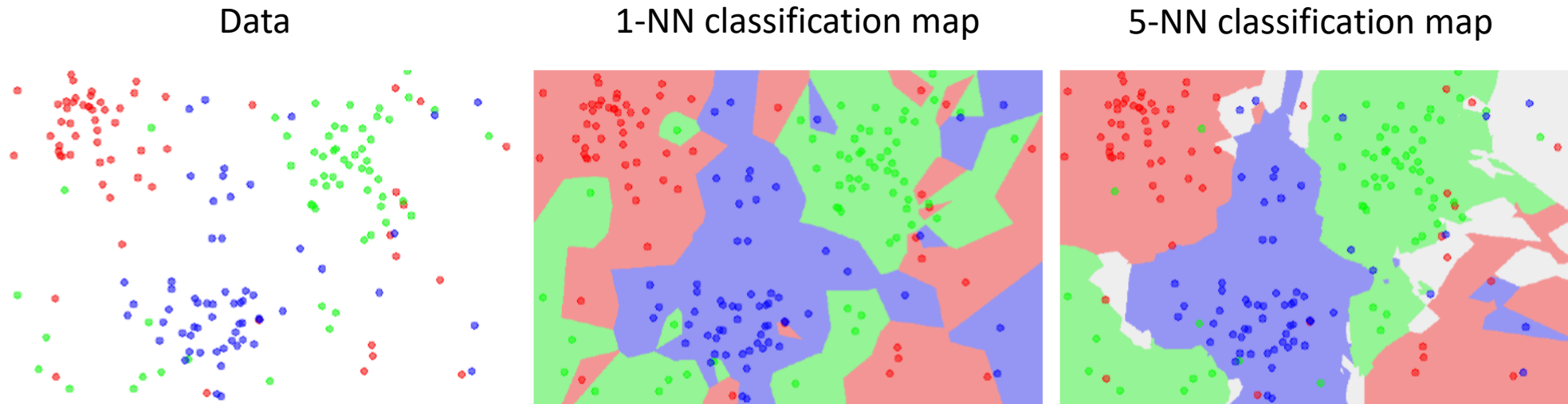
A Simple Classifier: Nearest Neighbor



Source: <https://commons.wikimedia.org/wiki/File:Data3classes.png>

Source: Niessner and Leal-Taixé, Introduction to Deep Learning

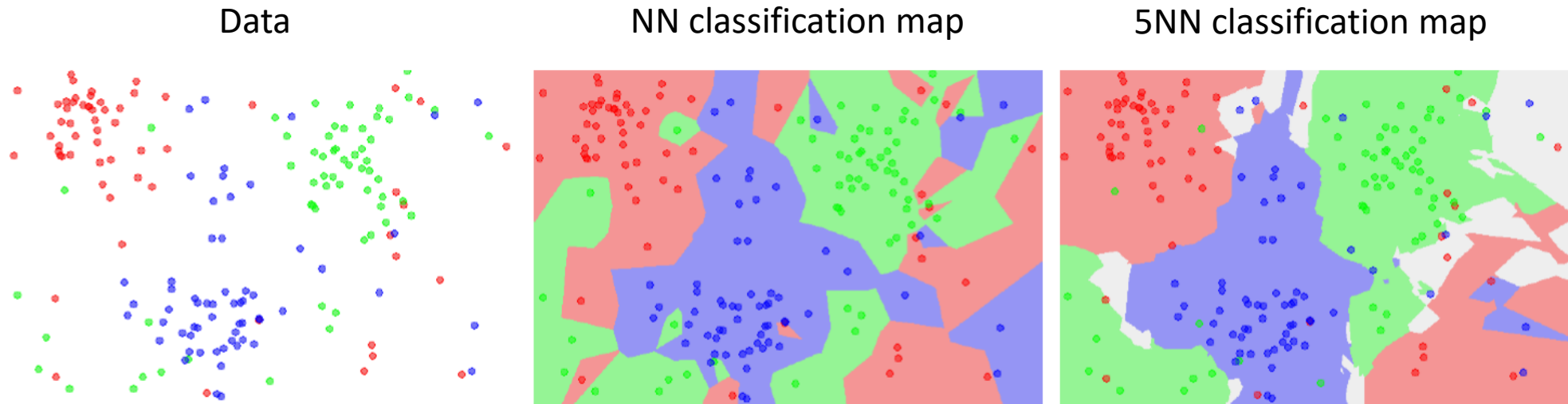
A Simple Classifier: Nearest Neighbor



Source: <https://commons.wikimedia.org/wiki/File:Data3classes.png>

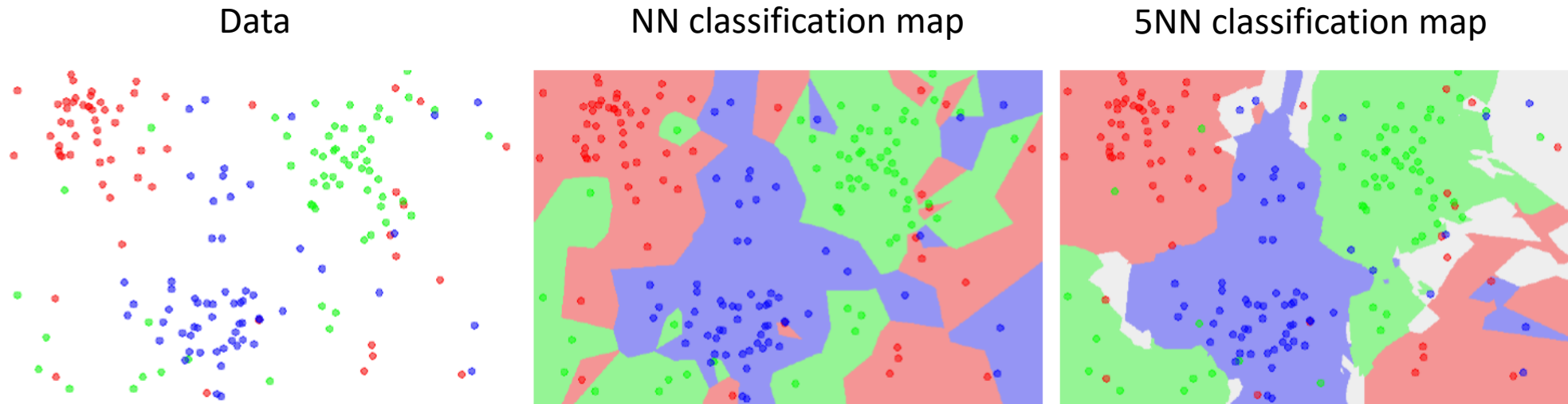
Source: Niessner and Leal-Taixé, Introduction to Deep Learning

A Simple Classifier: Nearest Neighbor



How does the NN classifier perform on training data?

A Simple Classifier: Nearest Neighbor



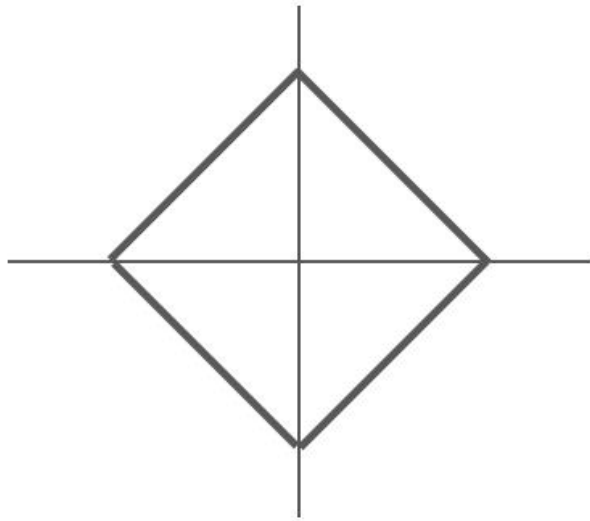
How does the NN classifier perform on training data?

What classifier is more likely to perform best on test data?

A Simple Classifier: Nearest Neighbor

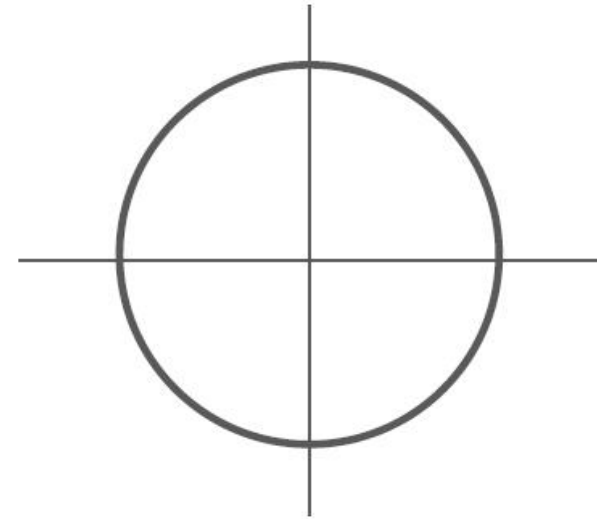
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (Euclidean) distance

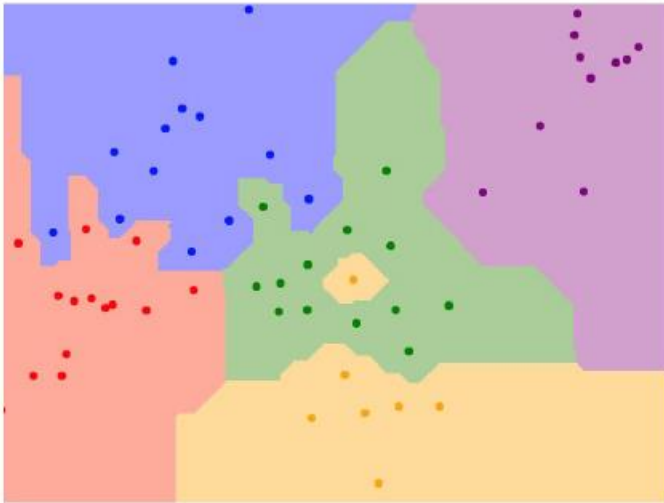
$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



A Simple Classifier: Nearest Neighbor

L1 (Manhattan) distance

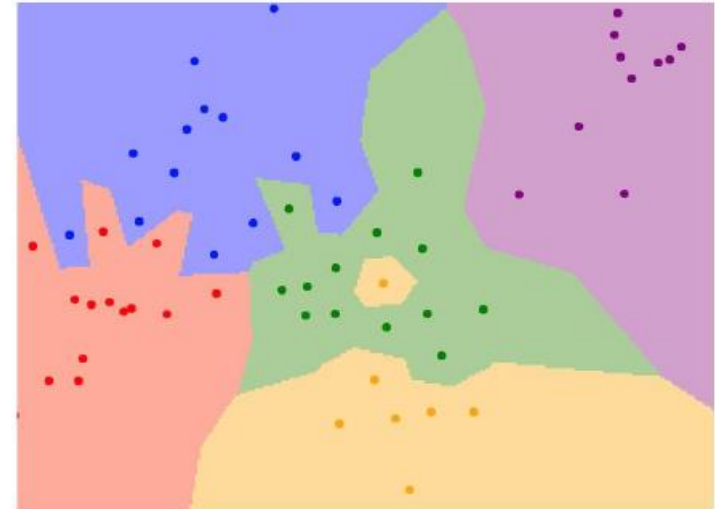
$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K = 1

<http://vision.stanford.edu/teaching/cs231n-demos/knn/>

Hyperparameters

- What is the best value of **k** to use?

Hyperparameters

- What is the best value of **k** to use?
- What is the best **distance** to use?

Hyperparameters

- What is the best value of **k** to use?
- What is the best **distance** to use?
- These are **hyperparameters**: choices about the algorithms themselves.

Hyperparameters

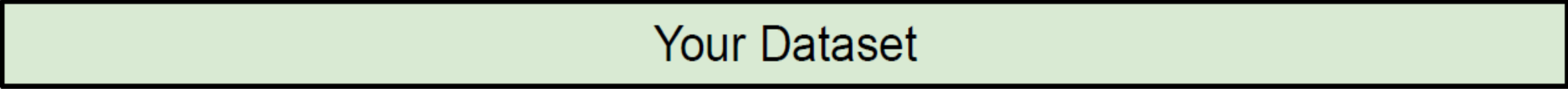
- What is the best value of **k** to use?
- What is the best **distance** to use?
- These are **hyperparameters**: choices about the algorithms themselves.

Very problem-dependent.

Must try them all out and see what works best.

Setting Hyperparameters

Idea #1: Choose hyperparameters
that work best on the data

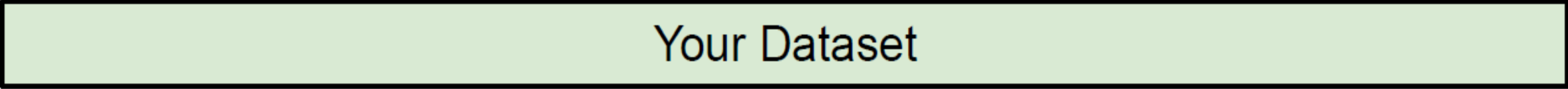


Your Dataset

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data

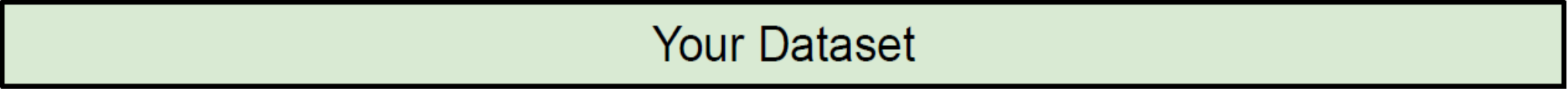


Your Dataset

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data



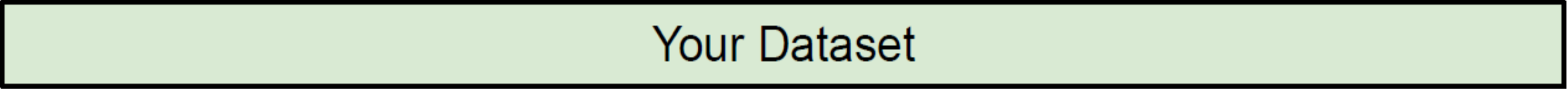
train

test

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



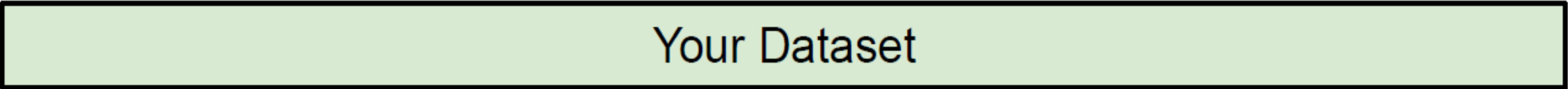
train

test

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

A single horizontal bar with a light green background and a black border, representing the entire dataset.

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



train

A horizontal bar divided into two sections. The left section is light green and labeled 'train'. The right section is light red and labeled 'test'.

test

Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test



train

A horizontal bar divided into three sections. The left section is light green and labeled 'train'. The middle section is light yellow and labeled 'validation'. The right section is light red and labeled 'test'.

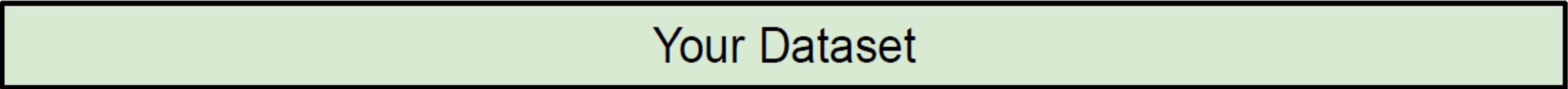
validation

test

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: $K = 1$ always works perfectly on training data



Your Dataset

A single horizontal bar with a light green background and a black border, representing the entire dataset.

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data



train

A horizontal bar divided into two sections. The left section is light green and labeled 'train'. The right section is light red and labeled 'test'.

test

Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

Better



train

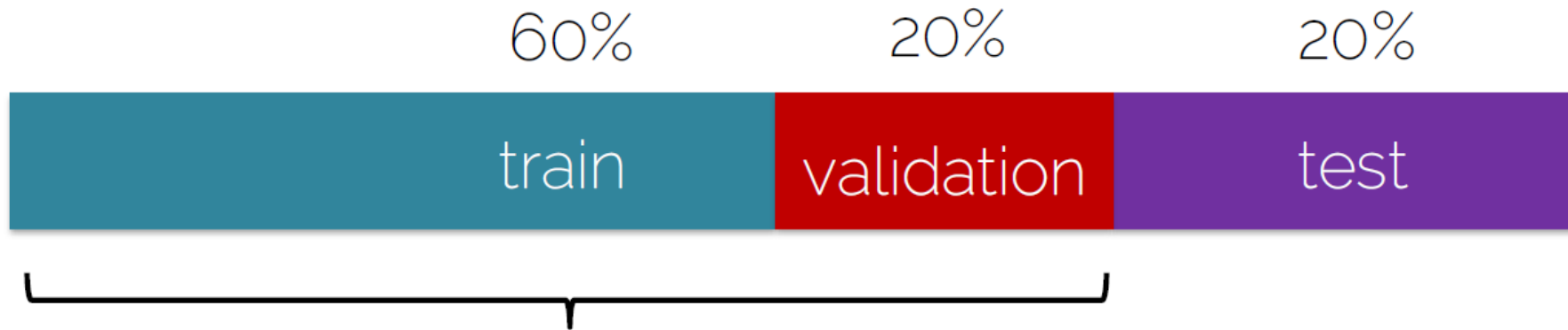
A horizontal bar divided into three sections. The left section is light green and labeled 'train'. The middle section is light yellow and labeled 'validation'. The right section is light red and labeled 'test'.

validation

test

Setting Hyperparameters

- Split your data



Find your hyperparameters

Other splits are also possible (e.g., 80%/10%/10%)

Setting Hyperparameters

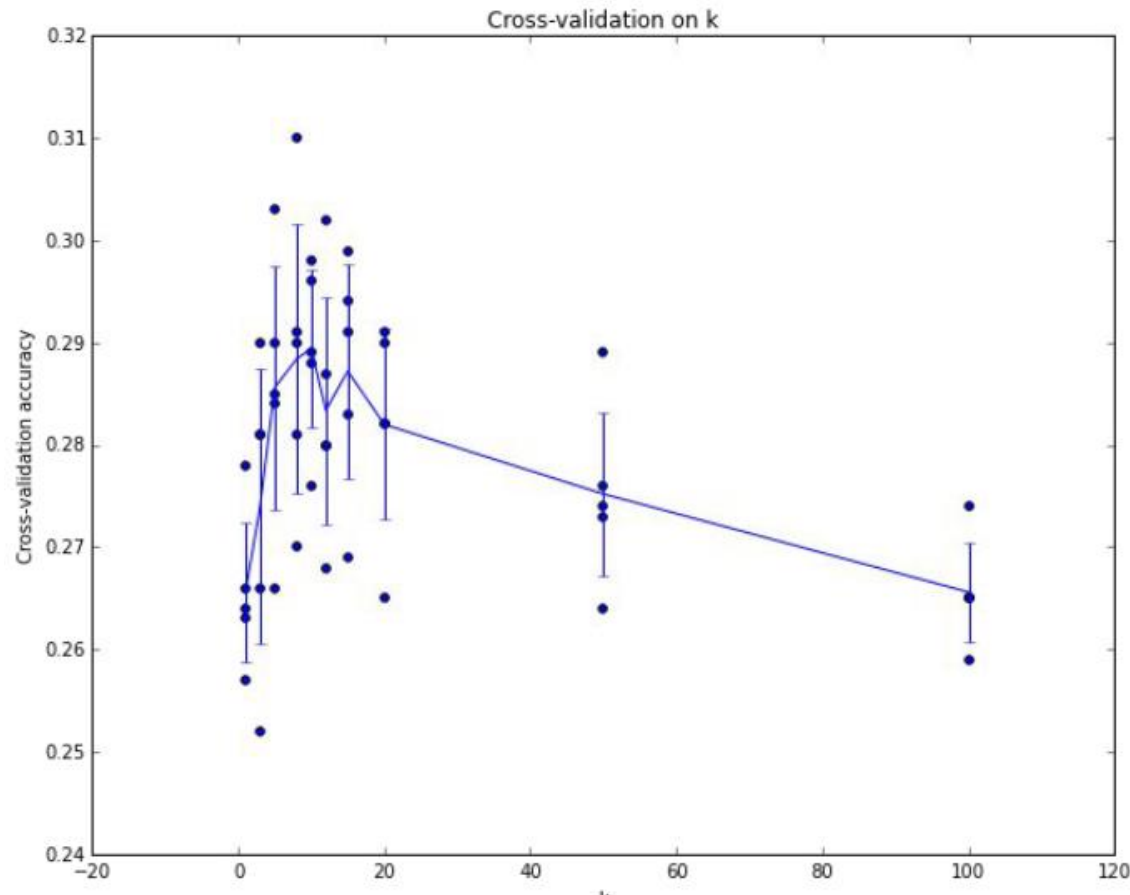
Your Dataset

Idea #4: Cross-Validation: Split data into **folds**, try each fold as validation and average the results

fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test

Useful for small datasets, but not used too frequently in deep learning

Setting Hyperparameters



- Example of 5-fold cross-validation for the value of k.
- Each point: single outcome.
- The line goes through the mean, bars indicated standard deviation

(Seems that $k \approx 7$ works best for this data)

k-Nearest Neighbor with pixel distance **never used**

- Distance metrics on pixels are not informative
- Very slow at test time

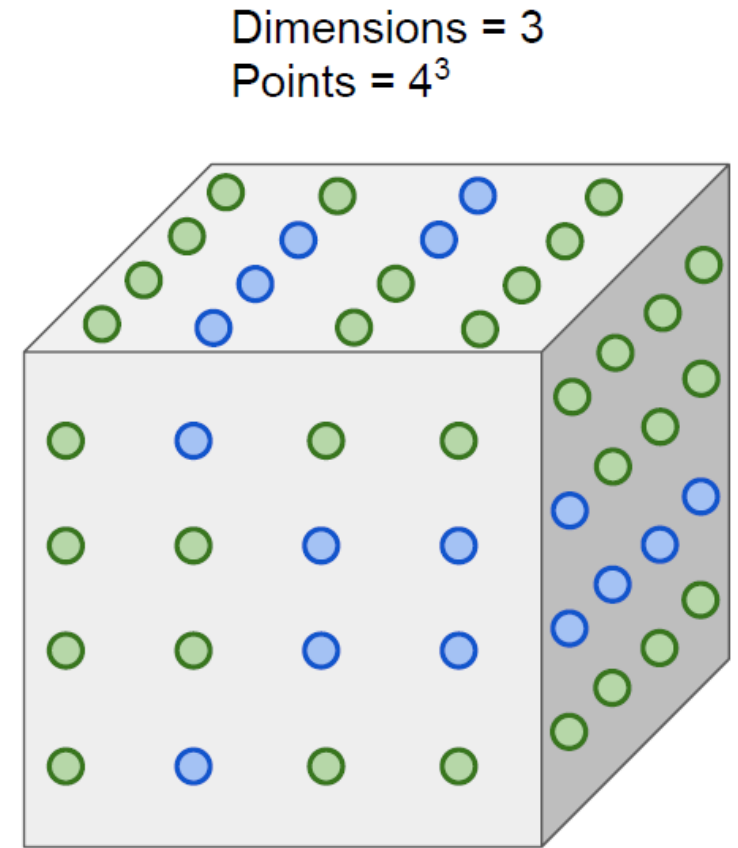
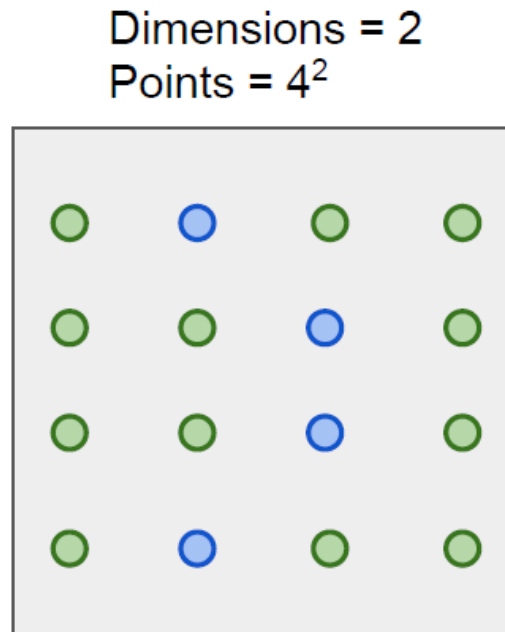
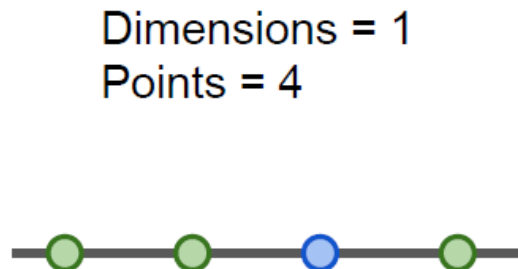


[Original image is
CC0 public domain](#)

(all 3 images have same L2 distance to the one on the left)

k-Nearest Neighbor with pixel distance **never used**

- Curse of dimensionality



K-Nearest Neighbors: Summary

- In **Image classification** we start with a **training set** of images and labels, and must predict labels on the **test set**
- The **K-Nearest Neighbors** classifier predicts labels based on the K nearest training examples
- Distance metric and K are **hyperparameters**
- Choose hyperparameters using the **validation set**;
- Only run on the test set once at the very end!
- Pixel distance is not very informative.